Automated Dynamic Analysis Signature Generation Using Large Language Models

Ikram Benfellah, Fadel Fatima Zahra

March 22, 2026

Abstract

This report documents the design, implementation, and evaluation of an automated system for generating CAPEv2 malware detection signatures from dynamic analysis reports using Large Language Models. The system processes sandbox execution logs through a six-stage pipeline: data acquisition, feature extraction, behavior representation, LLM-driven semantic analysis, signature synthesis, and validation. We evaluate the approach on 100 malware samples across 10 families (Adload, Emotet, HarHar, Lokibot, njRAT, Qakbot, Swisyn, Trickbot, Ursnif, Zeus), achieving 86% precision and 79% recall on original samples and 77% precision on cross-family generalization. A multi-stage validation pipeline catches approximately 15% of LLM-generated errors, resulting in production-ready signatures. We document technical challenges, engineering trade-offs, and lessons learned throughout the project lifecycle.

1 Research Objective and Motivation

Modern security operations centers face a persistent bottleneck: manual signature development is still time-consuming and labor-intensive. Analysts spend 4–6 hours per malware sample writing CAPEv2 behavioral signatures by extracting patterns from sandbox logs, translating those patterns into Python code, and validating the logic against real samples.

Our central research question is straightforward: *can Large Language Models automate signature generation while maintaining acceptable precision and operational utility?*

To address this question, we designed a six-stage pipeline:

1. Load and parse CAPEv2 dynamic analysis reports
2. Extract structured behavioral features from execution traces
3. Build a unified behavior intermediate representation (Behavior IR)
4. Apply LLM-based semantic analysis to identify malware families and behaviors
5. Generate CAPEv2 Python signatures automatically
6. Evaluate quality, validate correctness, and measure cross-family generalization

The approach combines traditional feature extraction with modern LLM capabilities to reduce manual analyst burden. Rather than replacing analysts, the system serves as an augmentation tool that produces initial signature drafts for human review and refinement.

2 System Design and Implementation

2.1 Data Acquisition and CAPEv2 Report Parsing

CAPEv2 sandbox reports are complex, deeply nested JSON structures containing comprehensive execution traces. Each report includes process activity, registry modifications, file operations, network traffic, API calls, and behavior classifications. In practice, CAPEv2 schemas vary significantly across samples, including inconsistent field hierarchies, optional fields, and different levels of recording granularity.

Our parsing strategy employed hierarchical, fault-tolerant extraction with extensive error handling and fallback mechanisms. Rather than assume a strict schema, we implemented conditional field extraction: queries check multiple plausible paths for each behavioral category, fall back to defaults when primary paths are absent, and log parsing anomalies for later review.

Key challenges in this phase included:

- **Schema inconsistency:** API call data nested under `behavior.apicalls` in some samples and `processes[].apicalls` in others
- **Sparse data:** Some samples recorded minimal activity; others produced megabytes of trace data
- **Malformed entries:** Occasional corrupt records required graceful degradation

Resolution required custom parsing code rather than off-the-shelf frameworks, as the latter could not accommodate the variation. This stage consumed more development effort than anticipated, underscoring the principle that real-world data is messier than academic datasets.

2.2 Feature Extraction and Behavioral Signal Identification

Extracting meaningful behavioral features required determining which aspects of execution traces are discriminative for malware family classification. Initial exploration considered 50+ candidate features (process creation counts, registry modification frequencies, API call distributions, network endpoint counts, etc.), but individual features proved insufficient for high-precision detection.

The breakthrough came from recognizing that malware behavior is best represented as *behavioral patterns*: contextual combinations of events rather than isolated actions. A benign application might create processes or modify registry keys, but malicious samples tend to show coordinated behavior chains. For example, the pattern *process injection + registry modification + outbound network activity* provides a much stronger signal than any individual component.

We therefore structured feature extraction around behavioral groups:

- **Process operations:** Process creation, termination, injection, privilege escalation
- **Registry operations:** Modification of registry keys, particularly those associated with persistence (Run keys, Services)
- **File operations:** File creation, modification, deletion, with emphasis on dropped files and system directories
- **Network indicators:** DNS lookups, TCP connections, HTTP/HTTPS traffic, particularly to external endpoints
- **API call patterns:** Sequences of system calls grouped by function category

These groups capture behavioral intent rather than mechanical event counts, enabling more precise malware characterization.

2.3 Behavior Intermediate Representation

Raw CAPEv2 JSON is unsuitable for direct LLM processing. The nested structure, schema variation, and irrelevant detail introduce excessive noise. In our experiments, feeding complete JSON reports increased token consumption by $\sim 50\%$ and degraded output quality.

We therefore designed the Behavior Intermediate Representation (Behavior IR): a simplified JSON format that abstracts CAPEv2 complexity while preserving semantic content. The Behavior IR structures data as:

- **Header:** Sample metadata (SHA256, compile time, original filename)
- **Behavioral sections:** Groups of related behaviors (process hierarchy, registry modifications, network connections)
- **Annotated patterns:** Behavioral chains with confidence scores

This abstraction reduced JSON size by $\sim 40\%$, cut token usage in half, and noticeably improved LLM analysis quality. The IR also served as a stable interface between upstream parsing and downstream analysis, enabling easier testing, caching, and debugging.

2.4 LLM-Driven Semantic Analysis

For semantic interpretation of behaviors, we selected Llama 2 (7B), deployed locally to avoid API costs and preserve data privacy. This choice trades inference speed ($\sim 4 - -6$ seconds per sample) for zero per-sample cost and full reproducibility.

Our LLM integration employed two-level analysis:

1. **Per-sample analysis:** Detailed semantic interpretation of individual sample behaviors, family hypothesis, and characteristic patterns. This produces a detailed behavior summary for each sample.
2. **Per-family aggregation:** Across all samples in a family, synthesize common behavioral patterns, identify family-specific signatures, and generate family-level profiles

The two-level structure proved superior to single-pass analysis. Attempting to analyze all 100 samples in one global pass exceeded context limits, produced generic patterns, and wasted inference cycles. Per-family profiles gave more accurate signatures at lower inference cost.

Key optimizations included:

- **Aggressive caching:** All LLM results cached locally. Process crashes did not require restart of inference.
- **Batch processing:** Similar samples grouped for inference to exploit caching
- **Prompt engineering:** Iterative refinement of prompts to reduce hallucination and improve consistency

2.5 Automated Signature Generation and Validation

Generating CAPEv2 Python signatures from LLM analyses represents the critical transformation from semantic interpretation to executable code. Direct prompt-based generation produced syntactically passable code with subtle logical errors.

We implemented a three-stage validation pipeline:

1. **Syntax validation:** Python AST parsing confirms syntactic correctness; missing imports and malformed statements are caught immediately
2. **Logical validation:** Behavioral pattern checker verifies that signature logic matches the Behavior IR (e.g., no contradictory conditions, proper API sequencing)
3. **Functional testing:** Signatures executed against original sample to confirm detection

The validation pipeline caught approximately 15% of generated signatures (3 out of 20), preventing erroneous deployments. Common errors included:

- Missing Python imports (json, re, ctypes)
- Incorrect CAPEv2 API method calls (e.g., calling methods that don't exist)
- Logic errors in pattern matching (contradictory conditionals, unreachable code)

Failed signatures were reprocessed by the LLM with corrective feedback: "Your signature failed syntax check: missing import re. Regenerate with proper imports." This feedback loop improved regeneration success to near 100%.

2.6 MITRE ATT&CK Technique Mapping

Beyond signature generation, we mapped detected behaviors to MITRE ATT&CK techniques, providing frameworks for threat intelligence and incident response. Initial fully automated mapping achieved only 68% accuracy, as the LLM made confident but incorrect mappings on subtle behavioral distinctions.

We added a confidence-aware filtering layer: mappings with LLM confidence below 75% were flagged for manual review rather than automatically committed. This hybrid approach resulted in 84% overall accuracy while preserving analysts' ability to catch edge cases. Discovery techniques, in particular, proved ambiguous in sandbox artifacts and were consistently marked for expert review.

3 Challenges, Failures, and Lessons Learned

3.1 Failed Approaches and Corrective Actions

3.1.1 Approach 1: Raw JSON to LLM

Initial hypothesis: Feed complete CAPEv2 JSON directly to Llama and request signature generation. **Outcome:** Unsuccessful. LLM output was incoherent, and pattern detection was ineffective. Approximately 50,000 tokens of nested JSON introduced too much noise. **Correction:** Designed Behavior IR to abstract complexity and reduce token consumption by 50%.

3.1.2 Approach 2: Monolithic Global Analysis

Initial hypothesis: Analyze all 100 samples in a single LLM pass, request synthesis of global patterns. **Outcome:** Unsuccessful. Context limits were exceeded, resulting in generic patterns that missed family-specific behavior and used resources inefficiently. **Correction:** Two-level analysis (per-sample → per-family) dramatically improved quality and reduced inference cost.

3.1.3 Approach 3: Fully Automated MITRE Mapping

Initial hypothesis: LLM generates MITRE mappings without human review. **Outcome:** 68% accuracy on technique classification, with confident but systematic misclassifications on subtle behavioral distinctions. **Correction:** Introduced confidence thresholds (75%+) and mandatory manual review for low-confidence predictions. Accuracy improved to 84%.

3.2 Timeline Reality Check

Our original project timeline estimated 2 weeks; actual execution required 4 weeks. The divergence reflects underestimation of real-world complexity:

Phase	**Estimated**	**Actual**
Data parsing and CAPEv2 wrangling	1 week	2 weeks
Feature extraction	1 week	5 days
Behavior IR design	3 days	2 days
LLM integration and optimization	1 week	10 days
Signature generation and validation	3 days	1 week
Evaluation and cross-validation	3 days	5 days
Documentation and write-up	2 days	4 days
Total	2 weeks	4 weeks

Data handling consumed significantly more time than anticipated. JSON parsing, schema variations, and error recovery became major focuses. Security-specific validation requirements (the three-stage signature pipeline) also consumed more effort than initial estimates.

3.3 What Worked Well

- **Result caching:** Saved estimated 2 days of work by enabling process resumption after crashes
- **Intermediate representations:** Behavior IR abstraction made the system maintainable and testable
- **Modular notebook structure:** Six separate notebooks enabled parallel development and simplified debugging
- **Multi-stage validation:** Rigorous validation pipeline caught errors early, preventing deployment of faulty signatures
- **Two-level LLM analysis:** Per-family aggregation substantially improved signature quality and reduced inference cost

3.4 What We Would Do Differently

- **Prototype with smaller dataset:** 100 samples was large for rapid iteration. Starting with 10 samples to validate the pipeline, then scaling, would have accelerated development
- **Lock down data format specification early:** Spending upfront effort on comprehensive CAPEv2 schema documentation would have reduced parsing complexity
- **Implement manual review gates from day one:** We flagged low-confidence predictions late in development. Earlier integration of human review would have improved downstream quality
- **Budget inference costs explicitly:** Although using local Llama avoided API costs, explicit budgeting for inference time would have motivated earlier optimization

4 Technical Results and Evaluation

4.1 Signature Generation and Syntax Validation

We successfully generated 20 CAPEv2 Python signatures (2 per family across 10 families). All signatures passed syntax validation and functional testing:

- **Generation success rate:** 100% (all 20 signatures syntactically valid after validation pipeline)
- **Average signature size:** 45 lines of Python code
- **Functional coverage:** All signatures tested against original samples
- **Deployment status:** Production-ready

The validation pipeline rejected 3 signatures (15%) before initial deployment, and all reprocessed signatures passed subsequent validation.

4.2 Detection Quality and Precision-Recall Trade-offs

We evaluated signature quality using two complementary metrics:

Internal validation (detection against original samples):

- Precision: 86%
- Recall: 79%
- F1-Score: 0.82

The 14% false positive rate stems from over-generalized pattern matching. We could reduce this by tightening pattern thresholds, but that would degrade recall. The current operating point (86% precision / 79% recall) reflects a reasonable balance for production SOC deployment.

Cross-family generalization (signatures against held-out test family):

- Precision: 77%
- Recall: 65%

The 14% precision decrease and 21% recall decrease on unseen families reflect the family-specific nature of some behaviors. However, 77% precision still indicates meaningful generalization: signatures trained on 9 families can detect behaviors in held-out families at usable rates.

4.3 MITRE ATT&CK Mapping Accuracy

We evaluated LLM-generated MITRE technique mappings against manual annotation across 100 behavioral samples:

Technique Category	**Accuracy**
Execution (Process Creation, Script Execution)	91%
Persistence (Registry Run Keys, Services, Scheduled Tasks)	88%
Command & Control (Outbound C2 Communication)	86%
Discovery (Process/Service/Network Discovery)	68%
Overall	84%

The asymmetry is expected: explicit behaviors (process creation, C2 connections) map reliably; implicit signals (discovery behaviors) require analyst judgment. The 84% overall accuracy, coupled with confidence-aware flagging of low-confidence predictions, represents a practical balance.

4.4 Operational Impact and Time Savings

In practical terms, the system achieves substantial analyst time reduction:

- **Manual signature writing:** 4–6 hours per sample
- **LLM-assisted draft:** 15–20 minutes per sample (data processing + LLM analysis)
- **Validation and refinement:** 40–50 minutes per sample
- **Total assisted workflow:** $\sim$1 hour per sample (80% time reduction)

The analyst role shifts from signature authorship to curation and validation, a qualitative change that increases throughput while maintaining security guarantees.

5 Honest Assessment: What Works and What Doesn't

5.1 Strengths

The core hypothesis is validated: LLMs can generate functional malware signatures at acceptable precision. Our 86% precision on original samples and 77% precision on unseen families demonstrates viability.

For security teams processing high sample volumes, this tool enables significant efficiency gains. Instead of 4–6 hours to hand-write a detection, analysts can validate an auto-generated signature in $\sim$1 hour. Over 100 samples annually, this represents 300–500 hours of freed analysis capacity.

5.2 Limitations

1. **Sandbox-constrained observations:** Detection capability is limited by what the sandbox observes. Polymorphic code, packed binaries, or behaviors that avoid sandbox triggers (e.g., ransomware that only modifies local files) will exhibit reduced detectability.
2. **Static sample generalization:** Signatures are trained and evaluated on fixed samples. Real-world malware variants, polymorphic samples, or obfuscated derivatives might evade generated signatures. (Note: this limitation applies equally to manual signatures.)

3. **Implicit behavioral detection:** Detection of implicit malware intent (e.g., Discovery activities, which manifest as subtle behavioral patterns) remains challenging for automated systems and requires expert review.

4. **False positive trade-off:** The 14% false positive rate, while acceptable, requires operational tuning for specific SOC environments.

6 Technical Contributions

This work contributes several artifacts and insights:

1. **Behavior Intermediate Representation (Behavior IR):** A reusable abstraction layer that decouples raw CAPEv2 data from downstream analysis. The IR is useful for future tools, enables easier testing, and reduces token waste in LLM processing.

2. **Production-ready signature pipeline:** Twenty validated CAPEv2 Python signatures across 10 malware families, demonstrating end-to-end feasibility of automated signature generation.

3. **Multi-stage validation framework:** A three-gate validation approach (syntax, logic, functional) that is generalizable to other code generation tasks and essential for security-critical automation.

4. **Practical LLM integration in security:** Addresses real-world challenges (cost, latency, hallucination) in deploying LLMs for security workflows.

5. **Evidence of cross-family generalization:** Empirical validation that behavioral patterns generalize across malware families at useful accuracy levels.

6. **MITRE ATT&CK automation insights:** Systematic analysis of where LLM-based technique mapping succeeds and fails, informing hybrid human-AI threat intelligence workflows.

7 Conclusion

This project demonstrates the viability of LLM-assisted malware signature generation. The six-stage pipeline successfully transforms CAPEv2 sandbox reports into production-ready detection signatures with acceptable precision (86%) and demonstrates meaningful cross-family generalization (77% precision on unseen families).

The system is not a replacement for human malware analysts. Instead, it serves as an effective augmentation tool: automating routine pattern extraction and initial signature synthesis while preserving analyst judgment for edge cases, validation, and refinement. For organizations processing high malware volumes, the 80% time reduction per sample represents substantial operational value.

Key technical achievements include the Behavior Intermediate Representation (which abstracts CAPEv2 complexity and reduces LLM token usage), the multi-stage validation pipeline (which catches 15% of errors before deployment), and practical solutions to LLM deployment challenges (caching, batching, cost optimization).

Future work should focus on:

- Expanding the dataset to 500+ samples and 30+ families for improved generalization

- Fine-tuning Llama on security-specific corpora to improve behavioral semantic understanding
- Integrating real SOC tooling (SIEM, EDR) to close the feedback loop: deployed signatures $\rightarrow$ operational telemetry $\rightarrow$ signature refinement
- Improving low-accuracy domains (Discovery techniques, polymorphic detection) through hybrid approaches combining LLM reasoning with rule-based heuristics

Overall, the evidence supports cautious optimism. LLMs are useful in security automation only when paired with rigorous validation, intermediate abstractions, and analyst oversight. Without these safeguards, automation introduces risk; with them, it delivers practical security value.